



자율주행 차량 시점 영상에서의 Semantic Segmentation





목차 a table of contents

- 1 프로젝트 개요
- 2 데이터셋 & 모델 구조
- 3 학습 설정
- 4 결과 및 성능 향상 과정
- 5 주요 성과 & 한계점

팀원 소개



통계학과
21학번
김나현



휴먼기계바이오공학부
22학번
전예지



컴퓨터공학과
22학번
정은채

Part 1, 프로젝트 개요



자율 주행의 핵심 = 주변 환경 인식

차량은 주행 중 도로, 차선, 보행자, 차량, 신호등 등 다양한 객체를 실시간으로 인식해야 함.



Semantic Segmentation의 필요성

픽셀 단위 분류로 객체 경계를 정밀하게 파악, 자율주행에서 실제 활용 중, YOLO 등 검출 모델보다 더 섬세하고 직관적인 결과 제공

목표

자율주행 차량 시점 영상에서 도로/차선/보행자 영역을 정확히 분할하는 것!

Part 2, 데이터셋 & 모델 구조



Cityscapes

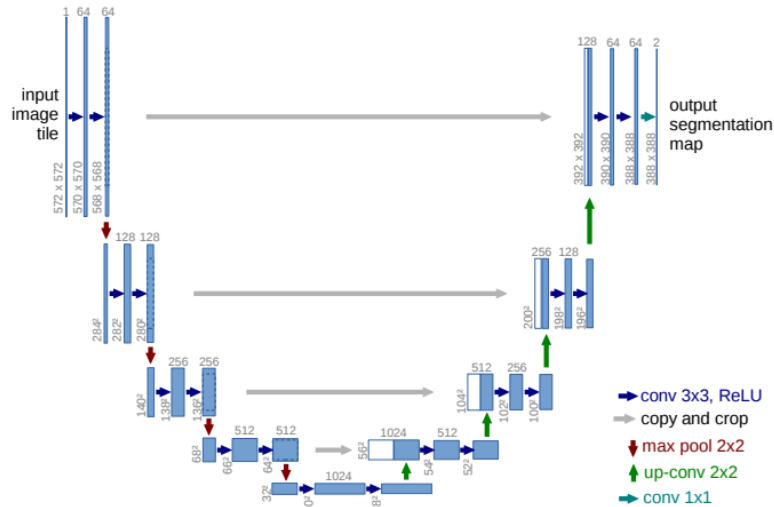
- 국가·지역
 - 독일 50개 도시에서 촬영
 - 주로 도심부, 교차로, 보행자 밀집 지역
- 환경 특성
 - 주간·맑은 날 중심
 - 보행자·차량·교통표지 등 도심 교통 요소가 풍부
 - 유럽형 도로 구조(좁은 차선, 보도와 차도 구분 뚜렷)
- 사용 데이터
 - [leftImg8bit_trainvaltest.zip](#) (5000 images)
 - [gtFine_trainvaltest.zip](#)
 - fine annotations for train&val sets (3475 annotated images)
 - dummy annotations (ignore regions) for test set (1525 images)

BDD100K

- 국가·지역
 - 미국 전역 (동부/서부/남부/북부)
 - 도심부, 교외, 고속도로 포함
- 환경 특성
 - 주·야간 모두 포함
 - 비·눈·안개 등 다양한 기상 조건
 - 미국형 넓은 도로 구조(넓은 차선, 신호등/표지판 위치 상이)
- 사용 데이터
 - [bdd100k_images_10k.zip](#)
 - 10,000장의 주행 장면 이미지
 - 전체 100K 영상에서 대표성 있는 샘플 10K 추출한 것.

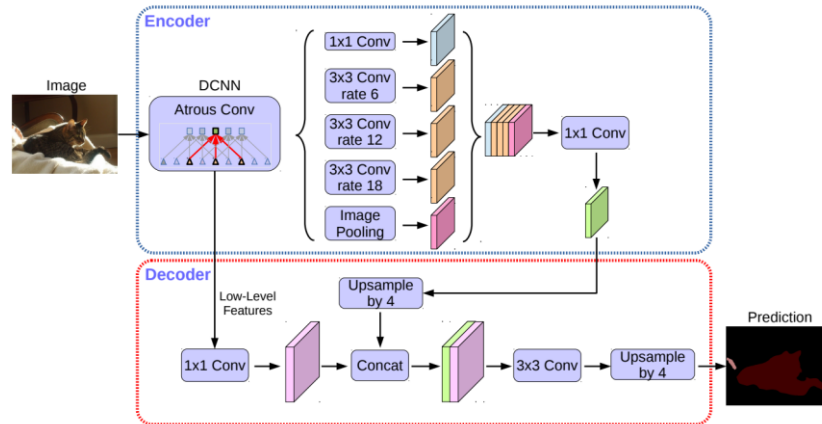
모델	발표연도 / 구조	주요 목적	장점 👍	단점 👎
U-Net	2015 / FCN 기반 U자형 Encoder-Decoder	생의학 이미지 정밀 분할	- 적은 데이터에도 강력 - 빠른 추론 속도	의료영상 특화 → 도로 환경에선 성능 제한
DeepLabV3+	2018 / FCN + Atrous Conv + Encoder-Decoder	경계선 정밀화 + 다중 스케일 문맥 포착	- 문맥 정보 추출 우수 - 경계 복원 강점	- 구조 복잡, 무거움 - 속도 느림, 메모리 사용량 큼
SegFormer	2021 / Transformer 인코더 + 경량 MLP 디코더	효율성과 정확성의 균형, 범용성	- 빠르고 정확 - 넓은 ERF 확보	극한 경량 디바이스 지원 불확실

U-Net



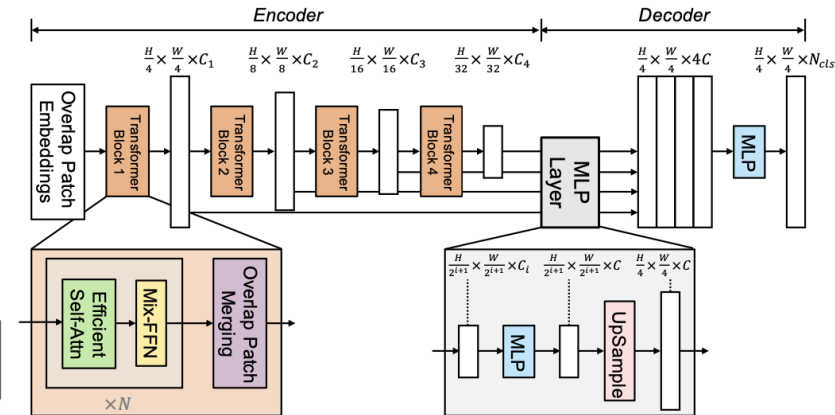
- **Contracting Path(수축 경로):** Conv+Pooling 반복 → 전역 문맥(context) 추출
- **Expansive Path(확장 경로):** Up-conv 반복 → 위치 정보 복원
- **Skip Connection:** 인코더 feature를 디코더로 직접 전달해 세밀한 경계 유지

DeepLab3+



- **Backbone:** ResNet-101, Xception 등
- **ASPP 모듈:** 다양한 dilation rate로 병렬 Atrous Conv → 멀티스케일 문맥 추출
- **Decoder:** 저해상도 semantic feature + 고해상도 edge/texture feature 결합 → 경계선 복원
- **Atrous Separable Conv:** 연산량 감소 및 속도 향상

Segformer



- **MiT Encoder:** Overlapping Patch Merging → 연속적인 local 정보 유지
- **Mix-FFN:** Positional encoding 없이 위치 정보 학습
- **Lightweight MLP Decoder:** 다중 스케일 feature 통합 → 넓은 ERF 확보
- **효과:** CNN 기반보다 더 넓고 유연한 문맥 정보 포착

Part 3, 학습 설정





- ✓ 손실 함수 : CrossEntropyLoss
- ✓ 옵티마이저 : Adam (lr=1e-5)
- ✓ Epoch : 10
- ✓ batch size : 2
- ✓ 입력 해상도 : 512 × 1024
- ✓ 디바이스 : CUDA (가능 시), 그 외 CPU
- ✓ Checkpoint : mIoU 기준 best 모델을 Google Drive에 자동 저장
- ✓ 클래스 수 : 19 (trainId 0-18)
 - 사용한 유효 클래스(original labelIds) 19개 지정
 - 매핑 : 위 labelIds → trainId 0-18로 리매핑
- ✓ 평가 지표 : Accuracy, mIoU (macro)

Part 4,

결과 및 성능 향상 과정



1. mIoU (Mean Intersection over Union)

- IoU (Intersection over Union) : 예측한 영역과 실제 정답(라벨) 영역이 얼마나 겹치는지를 나타내는 지표

⇒ mIoU는 모든 클래스에 대해 IoU를 구하고, 그 **평균값**을 계산한 것

$$IoU = \frac{\text{True Positive}}{\text{True Positive} + \text{False Positive} + \text{False Negative}}$$

$$mIoU = \frac{1}{N} \sum_{i=1}^N IoU_i$$

- 각 클래스에 대해 예측된 영역과 실제 정답 영역의 겹치는 정도를 계산
- > 전체 클래스에 대해 평균을 내어 산출
- 클래스 간 불균형이 큰 경우에도 공정한 평가가 가능하다는 장점이 있어 semantic segmentation에서 널리 사용

2. Pixel-wise Accuracy (전체 픽셀 정확도)

- 전체 이미지에서 정확히 예측한 픽셀의 수 / 전체 픽셀 수
- 클래스 간 균형을 고려하지 않은 전체 성능의 총합

$$Accuracy = \frac{\text{Number of Correctly Predicted Pixels}}{\text{Total Number of Pixels}}$$

- 가장 직관적인 지표로, 전체 이미지에서 맞게 예측한 픽셀의 비율을 나타냄
- 단점 : 자주 등장하는 클래스(예: 도로, 하늘)에 Accuracy가 치우쳐져 있을 수 있음

Pixel-accuracy
: 전체적인 성능 개요를 빠르게
파악 가능함

mIOU
: 각 클래스별 성능 편차를 반영해
공정하고 세밀한 평가 가능



두 지표를 병행하여
전반적인 예측 성능 +
클래스별 예측 품질
모두 평가

Part 4 각 데이터셋별 단일 모델 성능 비교

Accuracy	Cityscapes	BDD100k
UNet	0.5930	0.2177
Deeplabv3+	0.7200	0.5400
Segformer	0.7500	0.6200

mIOU	Cityscapes	BDD100k
UNet	0.1058	0.0058
Deeplabv3+	0.3041	0.2100
Segformer	0.4117	0.3000

[Soft voting]

각 모델이 출력한 softmax 확률값을 평균내어 최종 예측값으로 선택

$$\hat{y}_{pixel} = \arg \max_c \left(\frac{1}{M} \sum_{m=1}^M P_m^{(c)} \right)$$

- 모델의 Confidence 반영
- 클래스 예측 확률이 비슷할 경우 더 정밀한 판단이 가능함

[Hard voting]

각 모델의 예측 클래스 중 가장 많이 나온 값을 최종 예측값으로 선택

$$\hat{y}_{pixel} = \text{mode}([y_1, y_2, y_3])$$

- 예측 확률 정보는 무시함
- 신뢰도 정보는 반영 X

[Weighted Voting]

모델마다 중요도/신뢰도를 부여한 후 Hard / Soft Voting에 각각 반영함

- 모델 간 성능 차이를 반영함
- 잘하는 모델의 영향력을 키우고, 약한 모델은 줄일 수 0

Part 4 각 데이터셋별 앙상블 모델 성능 비교표

mIOU

mIOU	Cityscapes	BDD100k
Soft Voting	0.3085	0.0007
Hard Voting	0.3085	0.0014
Weighting Voting	0.3605	0.002

Accuracy

Accuracy	Cityscapes	BDD100k
Soft Voting	0.7277	0.0018
Hard Voting	0.7278	0.0018
Weighting Voting	0.7302	0.0072

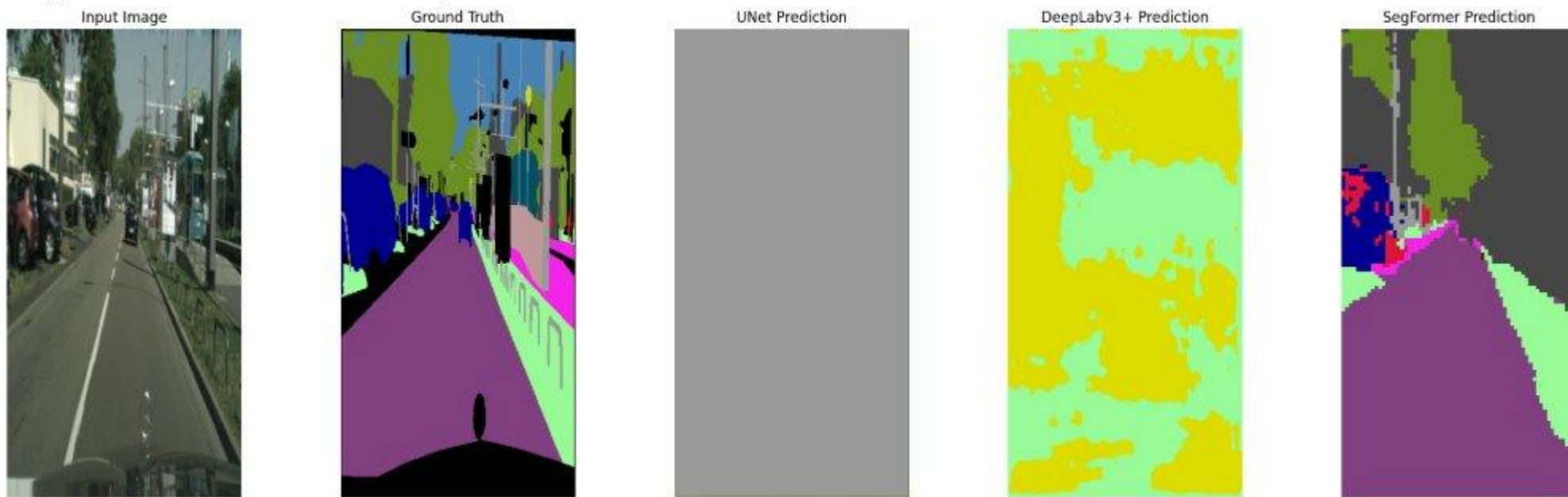
Part 4 클래스별 성능 차이 비교

[Image 1]

UNet - Acc: 0.0417, mIoU: 0.0030

DeepLabv3+ - Acc: 0.0443, mIoU: 0.0068

SegFormer - Acc: 0.5504, mIoU: 0.1738



임의의 이미지에 대해 비교해보면,

Unet의 경우 아예 객체 인식이 안되고 있는 상태

Deeplabv3+의 경우 클래스에 관계없이 크게 도로와 나머지로만 객체 인식하고 있는 상태

Segformer의 경우 3개의 모델 중 가장 성능이 좋음, 도로에 추가적으로 나무, 사람 등도 추가로 인식 성능



Part 5, 주요 성과 & 한계점

Part 5 주요 성과 요약 및 한계

< 성과 >

- ① 기존의 Unet, Deeplabv3+ 단일 모델과 비교하였을 때 3가지의 앙상블 기법들 모두에서 mIOU의 값이 향상됨
- ② Accuracy의 경우에도 3가지의 앙상블 기법들 모두에서 0.7 정도로 상당히 높은 성능을 나타냄
- ③ 앙상블 기법을 통해 단일 모델들의 단점 보완이 가능함

< 한계 >

- ① 기본 BDD100k 데이터셋의 경우 라벨링이 모호하여 Cityscapes 데이터셋과 비교하였을 때 전체적인 성능이 낮게 나옴
- ② 앙상블 기법이 무조건적인 성능 향상을 유발하는 것은 아님 -> 오히려 단일 모델의 성능이 더 좋은 경우도 존재함
 - ➡ 예) BDD100k 데이터셋의 Soft-voting mIOU : 0.002 vs 단일 모델 mIOU : 0.0007, 0.0014, 0.002

< 한계점 개선 방안 >

- ① 대안1 : 실시간 처리 제한
- ② 대안2 : BDD100k 데이터셋 자체의 모호한 라벨링을 보완하여 입력
- ③ 대안3 : Meta-Learner (MLP 앙상블 기법) 추가 실험

< 결론 >

- ① 2개의 데이터셋 모두에서 Segformer 모델의 성능이 최고
- ② 앙상블을 통한 성능 향상 가능성의 확인
- ③ 실제 자율주행 적용 등에 적용 시 확장성 있음

THANK YOU :)